

Team Name: Ranger Sharks

Names and NetIDs: James Cenawood (jcc463), Orion Hoch (ogh6), Max Masleyev (mam759), Emily Wei (ejw235)

Vision

Out of the three options from our previous report, we decided to work on the “Drop the Handkerchief game.” During this check-in, our group focused on building a version of the game that works in a two-player setup with a functioning and polished terminal UI. By creating a two-player version with a functioning backend and TUI for the check-in, we can then feel confident about diving into the AI opponent version of the game.

Summary of Progress

We split our work into two main areas: the backend game engine and the terminal UI. The backend team implemented the full game logic. This includes correct time logic and cylinder accumulation logic/bookkeeping for each player over the entire game and an abstract game type consumed by all helper/engine functions. The main large function we worked on was to resolve a half round which depended on if it resulted in a successful check, and a death after either a failed check or an overflow of the checking player’s cylinder. The UI team built a complete terminal interface using the Notty library, featuring animated player portraits and multiple game screens guiding players through each round, among other widgets and features. By the end of this milestone the game is fully playable from start to finish.

Activity Breakdown

Backend game design: James and Emily

- James:
 - Implemented the game.mli and [game.ml](#) file abstract type for game type and half round record in order to provide snapshots of each round for the history of the game and display
 - Implemented helper functions for accessing key game information from inside the abstract type
 - Threaded config parametrized helpers to take only a game as argument so that raw configs and player_states aren’t consumed in other modules
 - Worked with Emily to implement helper functions and overall half round play function to resolve each type of half round and advance the game clock properly
 - Added placeholder for the Hal AI and Hal.mli interface so we could baseline test if his actions in the actual series worked with the engine first

- Emily: I implemented the test cases for this project in the files test_game.ml and test_hal.ml using the module Alcotest. In the test_game.ml file, I added test cases that checked the drop and check time boundaries, the clock procedures, overall game progression, as well as role assignment for the default and Baku configurations. In test_hal.ml, I tested the lookup table where rounds 1-9 returned Some and rounds 10+ returned None, the full walkthrough of all 18 half-rounds, a bounds invariant check confirming that Hal's actions stayed between 1 and the turn_duration for all 24 half-rounds, and the fallback behavior.

Terminal UI: Orion and Max

- Orion: Created base modules for TUI and set up 'notty' library system to implement half blocks into the game screen. Wrote the starting text and connected inputs to animation screens. Made TUI layout and indexing screen to catalog location and number of elements.
- Max: I added some additional TUI widgets to complement some of the existing interfaces developed by Orion. Specifically, I implemented a text-based NDD progress bar to display each player's cylinder fill level, as well as a game archive feed showing a readable summary of recent turns. These widgets were integrated into bin/main to show up with the TUI, and as I worked on that I fixed several spots where the TUI code was out of sync with updates made to the game engine (such as function arguments changes).

Productivity Analysis

Overall, we made considerable progress towards this milestone and are happy with where we ended up. Dividing into a backend team and a UI team worked well. Both sides were able to make progress independently without getting in each other's way. Any of the small compatibility issues between the game engine TUI code were quickly and smoothly addressed. Our goals were realistic and we were able to deliver a playable two-player game by this milestone. Going into MS3, our main focus will be implementing the AI opponent, which should be more manageable now that the game engine and interface are both solidly in place.